

1. History of java:

The concept of developing java programming language was proposed by James Gosling, Patrick Naughton and Mike Sheridan, the member of green project, at sun microsystem in 1991. This language was initially called "Oak" but was renamed later to "Java" in May 1995. Many more people contributed to the design and evolution of this language. The primary motivation for the development of this language was the need for platform independent (i.e. architecture neutral) language that could be used to produce code that would run different environments. Java derives much of its character from c and c++.

2. Introduction to Java:

Java is one of the most widely used object oriented programming language which is used from software development to web development. It is purely object-oriented programming language. The two main components of java platform are Java API, which is library of java command lines and the JVM (Java Virtual Machines) that interprets the java code into machine code.

3. Features of Java:

- **Simple:** It was designed to be easy to use and learn. In Java, there are a small number of clearly defined ways to perform a given task.
- **Object-oriented:** It is purely object-oriented programming language. Everything is defined inside the class in the java. Even the main method is defined inside the class. It deals with class and object. **Class** is a user defined datatype which contains entire set of data and code (method) to manipulate the data. It is the collection of objects of similar type. **Objects** are the basic runtime entities in object oriented system. It is an instance of class programming problem is analyzed in terms of object and nature of communication between them. When a program is executed, object interacts with each other by sending messages without knowing details of their data or code.
- **Platform independent (Architecture neutral):** It is platform independent programming language. It has ability to run the same program on different system and easily move from one computer to another computer. The java compiler generates a bytecode, which is machine independent and can run on any environment (or platform) that contains the Java Virtual Machine (JVM). JVM is an interpreter for bytecode.
- **Highly secure:** Java is highly secured programming language. Every java program is run within JVM which prevents the program from generating side effects outside the system. In java bytecode verifier examines the bytecode to ensure that they are valid and do not affect the system. Various access restriction (such as private, public) are enforced in java which provides security.
- **Compiled and Interpreted:** It is both compiled and interpreted language. The compiler generates a byte code from java source code. The byte code is machine independent code which is able to run on any machine that has JVM. JVM interprets the bytecode and generates machine code that can be directly executed by machine which is running the java program.
- **Java is highly case sensitive language.**
- **Robust:** It means reliable. Java checks the code at compile time to find out errors early and also checks the code at run-time. Java is robust because it protects our computer from

bad program, provides portability and automatic memory management and garbage collection.

4. Implementation of java programs:

- **Creating a java program (source code):** Type the java code using editor such as notepad, netbeans, eclipse, etc. and then save it on a secondary storage device with .java extension.
- **Compiling a java program into bytecodes:** Use javac command to compile a program. The compiler converts the java source code into bytecodes. The bytecodes are platform independent and hence are portable. The bytecodes are .class files.
- **Loading a program into memory:** The class loader loads the .class files from disk to primary memory before it executes.
- **Bytecode verification and execution:** The bytecode verifier examines the bytecodes and then JVM interprets the bytecode and generates machine code which runs in specific machine.

Example:

```
public class simple
{
    public static void main(String args[]) {
        System.out.println("hello world");
    }
}
```

- To run this program, we need to save it as simple.java and compile it as below;
javac simple.java
- It generates simple.class (bytecode) file. To interpret this we use,
java simple
- Finally, this program gives the output
hello world

// A program to print the sum of two numbers

```
public class sum
{
    public static void main(String args[]) {
        int a=5, b=2;
        int c=a+b;
        System.out.println("sum="+c);
    }
}
```

5. Data types:

- **Integers:** Java defines four integer types; byte, short, int and long. Byte is the smallest signed integer type of 8-bit, short is a signed 16-bit type and is least used, int is signed 32-bit type and most commonly used and long is a signed 64-bit type.
- **Floating-point types:** Floating point numbers, also known as real numbers are used when evaluating expression that require fractional portion. It includes 'float' and 'double'. 'Float' is 32-bit and 'double' is 64-bit.

Example:

```
public class floatExample
{
    public static void main(String args[]) {
        double pi, r, a;
        r=4.5;
        pi=3.1416;
        a=pi*r*r;
        System.out.println("Area of circle is:"+a);
    }
}
```

// A Program to find the total seconds in 10 days.

```
public class IntExample
{
    public static void main(String args[]) {
        int days=10;
        long seconds;
        seconds=days*24*60*60;
        System.out.println("Total seconds in 10 days="+seconds);
    }
}
```

- **Characters:** The datatype that is used to store characters is 'char' which is 16-bit type.

```
public class charExample
{
    public static void main(String args[]) {
        char c1,c2;
        c1= 89;
        c2= 'A'
        System.out.println("c1 and c2 has values:"+c1+" "+c2+"respectively");
    }
}
```

- **Boolean:** Boolean is simple type & is represented by datatype "Boolean". It contains only of two possible values true or false.

Example:

```
class boolexample
{
    public static void main (String args[])
    {
        boolean b;
        b=true;
        System.out.println("b is"+b);
        b=false;
        System.out.println ("b is"+b);
        System.out.println("5<2 is" +(5<2));
    }
}
```

Here, $5 < 2$ uses ' $<$ ' as relation operator which returns the value "false". The extra parenthesis is used around $5 < 2$ because '+' operator has a higher precedence than $<$.

6. Variables:

Variables are the name of memory space used to store values during program execution. A variable is defined by a combination of a data type, an identifier (variable name) and an optional initializer that contains value. Its syntax is,

type identifier=value;

To declare more than one variable of the specified type, we can use a comma.

Example:

Int a; //declaration of variable a

Int b=2;c=3; //initialization of variable b & c with values 2 and 3 respectively

- **Dynamic initialization of variable:** Dynamic initialization of variable means that we can initialize the variables using expression. The value of the dynamic variable changes during compile time if the values in the expression changes.

Example:

Program to find square of any given value,

Public class example

{

Public static void main (String args[])

{

Int root=10; //root has a constant value assigned has a constant value assigned

Int square=root*root; //square as root*root expression.if we change value of root,the value of square changes accordingly after each compilation.

System.out.println("root="+root+ "and square=" +square);

}

}

The scope and lifeline of variables

The scope of a variable defines the section of the code in which the variable is visible. The lifetime of a variable refers to how long the variable exists before it is destroyed (deallocated). The scope of a variable defined by a method begins with its opening curly brace. If a method has parameters, than they are also included within method's scope. A variable declared inside a scope are not visible to code outside that scope.

Scopes can also be nested i.e. there can be inner and outer scopes. The objects declared in the outer scope will be visible to the code within the inner scope but the objects declared within the inner scope will be visible.

Example:

class scope {

public static void main(String args[]) {

Int a=5; b=3;

If(a>b) {

Int c=7;

System.out.println("the value of a ,b&c are"+a+";"+";" +c);

}

System.out.println("the value of c that cannot be printed is"+c);

} }

7. Operators:

- **Basic arithmetic operators:** Basic arithmetic operators includes addition (+), subtraction (-), multiplication (*), division (/). These can be applied to both integer & floating point type.

Example:

```
class basicoperator {  
    public static void main(String args[]) {  
        int a=2+3;  
        int b=a*2;  
        int c=b/5;  
        int d=c-1;  
        System.out.println("addition result a="+a);  
        System.out.println("\n multiplication result b="+b);  
        System.out.println("\n division result c="+c);  
        System.out.println("\n subtraction result d="+d);  
    } }
```

- **Modulus operator:** Modulus operator returns remainder of a division operator. It can be applied to both integer & floating point type.

Example:

```
class modulus {  
    public static void main(String args[]) {  
        int a =45;  
        float b=63.26;  
        System.out.println("a mod 5="+a%5);  
        System.out.println("b mod 10=b%10);  
    } }
```

- **Arithmetic assignment operator:** Arithmetic assignment operator can be combined with an assignment operator which are called arithmetic assignment operator. It includes addition assignment (+ =), subtraction assignment (- =), multiplication assignment (* =), division assignment (/ =), modulus assignment (% =).

Example:

```
class arithmeticassignmet {  
    public static void main(String args[]) {  
        int b=1,c=3,d=4,e=14,f=6,  
        b+=5; //b=b+5  
        c-=1;  
        d*=2;  
        e/=7;  
        f%=4;  
        System.out.println("b="+b+"c="+c+"d="+d+"e="+e+"f="+f);  
    } }
```

- **Increment & decrement:** The ++ and – are increment and decrement operators. The increment operator increases its operand by one and decrement operator decreases its operand by one.

Example:

```
class IncDec {  
    public static void main(String args[]) {  
        int a=3,b=1,c,d;  
        c=a++; //puts value of a in c & then increments  
        d=++; //increments b by 1 & put that value in b  
        a++; //increments a by 1  
        b++; //increments b by 1  
        System.out.println("a="+a);  
        System.out.println("b="+b);  
        System.out.println("c="+c);  
        System.out.println("d="+d);  
    } }  
}
```

Output:

```
a=4  
b=2  
c=3  
d=2
```

➤ **Boolean Logical Operators**

The Boolean logical operators operate only on Boolean operands. All of the binary logical operators combine two Boolean values to form a resultant Boolean value.

&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

➤ **Relational Operators**

The relational operators determine the relationship that one operand has to the other. They determine equality and ordering. The relational operators are shown here:

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

8. Control Flow:

➤ **Java's Selection Statements**

Selection statements allow us to control the flow of our program's execution based upon conditions known only during run time. The various control statements are as follows:

- **If Statement:** if statement in java is used to perform the basic conditional test that evaluates a given expression for a Boolean value of true or false. It's general syntax is:

```
If(condition)
{ code to be executed
}
```

The code to be executed may be multiple statements, all contained within the braces, each ending with a semi-colon.

Example:

```
class Ifexample {
public static void main(String args[]) {
if(5>1)
System.out.println("yes 5 is greater than 1");
} }
```

- **If else Statement:** The else keyword can be used with an **if** statement to specify alternative code to be executed when the condition is false. It is known as conditional branching because the program will branch along a certain route according to the result of a test. Here is the general form of the **if else** statement:

```
if (condition)
{
Code to be executed if true;
}
else
{
Code to be executed if false;
}
```

Here, each *statement* may be a single statement or a compound statement enclosed in curly braces. The *condition* is any expression that returns a **boolean** value. The **if else** works like this: If the *condition* is true, then *statement1* is executed. Otherwise, *statement2* is executed. For example,

```
class Ifelseexample
{
public static void main(String args[])
{
if(5>1)
System.out.println("yes 5 is greater than 1");
else
System.out.println("The statement is false");
}
}
```

- **The if-else-if Ladder:** A common programming construct that is based upon a sequence of nested **ifs** is the *if-else-if ladder*. It looks like this:

```
if(condition)
statement;
else if(condition)
```

```
statement;  
else if(condition)  
statement;  
...  
else  
statement;
```

The **if** statements are executed from the top down. As soon as one of the conditions controlling the **if** is **true**, the statement associated with that **if** is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final **else** statement will be executed. The final **else** acts as a default condition; i.e. if all other conditional tests fail, then the last **else** statement is performed. If there is no final **else** and all other conditions are **false**, then no action will take place.

// Demonstrate if-else-if statements.

```
class IfElseLadder{  
    public static void main(String args[]) {  
        int month = 4; // April  
        String season;  
        if(month == 12 || month == 1 || month == 2)  
            season = "Winter";  
        else if(month == 3 || month == 4 || month == 5)  
            season = "Spring";  
        else if(month == 6 || month == 7 || month == 8)  
            season = "Summer";  
        else if(month == 9 || month == 10 || month == 11)  
            season = "Autumn";  
        else  
            season = "Bogus Month";  
        System.out.println("April is in the " + season + ".");  
    }  
}
```

Here is the output produced by the program:

April is in the Spring.

- **Switch Statement:** The **switch** statement is Java's multi-way branch statement. It provides an easy way to execute different parts of our code based on the value of an expression. It often provides a better efficient alternative than a large series of **if-else-if** statements. Here is the general form of a **switch** statement:

```
switch (expression) {  
    case value1:  
        // statement sequence  
        break;  
    case value2:  
        // statement sequence  
        break;  
    ...  
    case valueN:  
        // statement sequence
```



```
break;
default:
// default statement sequence
}
```

The *expression* must be of type **byte**, **short**, **int**, or **char**; each of the *values* specified in the **case** statements must be of a type compatible with the expression. Each **case** value must be a unique literal. Duplicate **case** values are not allowed.

The **switch** statement works like this: The value of the expression is compared with each of the literal values in the **case** statements. If a match is found, the code sequence following that **case** statement is executed. If none of the constants matches the value of the expression, then the **default** statement is executed. However, the **default** statement is optional. If no **case** matches and no **default** is present, then no further action is taken.

The **break** statement is used inside the **switch** to terminate a statement sequence. When a **break** statement is encountered, execution branches to the first line of code that follows the entire **switch** statement. This has the effect of “jumping out” of the **switch**.

A simple example of the switch:

```
class SampleSwitch {
public static void main(String args[]) {
for(int i=0; i<6; i++)
switch(i) {
case 0:
System.out.println("i is zero.");
break;
case 1:
System.out.println("i is one.");
break;
case 2:
System.out.println("i is two.");
break;
case 3:
System.out.println("i is three.");
break;
default:
System.out.println("i is greater than 3.");
}
}
}
```

The output produced by this program is shown here:

```
i is zero.
i is one.
i is two.
i is three.
i is greater than 3.
i is greater than 3.
```

Here, the statements associated with the **case** constant that matches **i** are executed. All others are bypassed. After **i** is greater than 3, no **case** statements match, so the **default** statement is

executed. The **break** statement is optional. If we omit the **break**, execution will continue on into the next **case**.

➤ **Iteration Statements:** Java's iteration statements are used to execute a block of code several number of times, and is often referred to as a loop. A loop repeatedly executes the same set of instructions until a termination condition is met. Java has very flexible three looping mechanisms. You can use one of the following three loops: while Loop, do...while Loop and for Loop.

- **While Statement:** The **while** loop is Java's most fundamental loop statement. It repeats a statement or block while its condition is true. It is an entry control loop. Here is its general form:

```
while(condition) {  
    // body of loop  
}
```

The *condition* can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true. When *condition* becomes false, control passes to the next line of code immediately following the loop. The curly braces are unnecessary if only a single statement is being repeated.

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int x = 10;  
        while( x < 20 ) {  
            System.out.println("value of x : " + x );  
            x++;  
            System.out.println("\n");  
        }  
    }  
}
```

This would produce the following result:

```
value of x : 10  
value of x : 11  
value of x : 12  
value of x : 13  
value of x : 14  
value of x : 15  
value of x : 16  
value of x : 17  
value of x : 18  
value of x : 19
```

- **do...while Loop:** A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time. It is an exit control loop. The syntax of a do...while loop is:

```
do  
{
```

```
//Statements
}while(condition;
```

The condition appears at the end of the loop, so the statements in the loop execute once before the condition is tested. If the condition is true, the flow of control jumps back up to do, and the statements in the loop execute again. This process repeats until the condition is false.

Example:

```
public class Test {
    public static void main(String args[]){
        int x = 10;
        do{
            System.out.println("value of x : " + x );
            x++;
            System.out.println("\n");
        }while( x < 20 );
    }
}
```

This would produce the following result:

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

- **for Loop:** A for loop is a repetition control structure that allows us to efficiently write a loop that needs to execute a specific number of times. It is also an entry control loop. A for loop is useful when we know how many times a task is to be repeated. The syntax of for loop is:

```
for(initialization;condition; update)
{
    //Statements
}
```

Example:

```
public class Test {
    public static void main(String args[]) {
        for(int x = 10;x < 20; x++) {
            System.out.println("value of x : " + x );
            System.out.println("\n");
        }
    }
}
```

This would produce the following result:

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

- **Jump Statements:** Java supports three jump statements: **break**, **continue**, and **return**. These statements transfer control to another part of our program.

The break Keyword: The *break* keyword is used to stop the entire loop. It can be used to terminate a statement sequence in a **switch** statement. The break keyword must be used inside any loop or a switch statement. The break keyword will stop the execution of the innermost loop and start executing the next line of code after the block. The syntax of a break is a single statement inside any loop:

```
break;
```

Example:

// Using break to exit a loop.

```
class BreakLoop
{
    public static void main(String args[])
    {
        for(int i=0; i<100; i++)
        {
            if(i == 10) break; // terminate loop if i is 10
            System.out.println("i: " + i);
        }
        System.out.println("Loop complete.");
    }
}
```

This program generates the following output:

```
i: 0
i: 1
i: 2
i: 3
i: 4
i: 5
i: 6
i: 7
i: 8
i: 9
```

Loop complete.

The continue Keyword: The *continue* keyword can be used in any of the loop control structures. It causes the loop to immediately jump to the next iteration of the loop.

- In for loop, the continue keyword causes flow of control to immediately jump to the update statement.
- In a while loop or do/while loop, flow of control immediately jumps to the condition.

The syntax of continue is a single statement inside any loop:

`continue;`

Example:

```
class Continue {
    public static void main(String args[]) {
        int i=0;
        while(i<6)
        {
            if(i==3)
                continue;
            System.out.println("Loop iteration is "+ i);
        } } }
```

Output:

```
Loop iteration is 0
Loop iteration is 1
Loop iteration is 2
Loop iteration is 4
Loop iteration is 5
```

return: The last control statement is **return**. The **return** statement is used to explicitly return from a method i.e. it causes program control to transfer back to the caller of the method. So, it can be categorized as a jump statement. In a method the **return** statement can be used to cause execution to branch back to the caller of the method. Thus, the **return** statement immediately terminates the method in which it is executed.

Example:

```
class Return {
    public static void main(String args[]) {
        boolean t = true;
        System.out.println("Before the return.");
        if(t) return; // return to caller
        System.out.println("This won't execute.");
    }
}
```

Output:

```
Before the return.
```

Here the final **println()** statement is not executed. As soon as **return** is executed, control passes back to the caller.

9. Array

An array is a group of contiguous or related data items that share a common name. It stores a collection of values of same type.

Obtaining an array is a two-step process:

- I. We must declare an array variable of the desired type as:

type arrayvariablename[];

or

type[] arrayvariablename;

eg: int a[];

or

int[] a;

- II. We must allocate the memory that will hold the array, using new operator and assign it to the array variable as:

arrayvariablename = new type[size];

eg; a=new int[10];

The array declaration can also be done in a single step as:

type arrayvariablename[] = new type[size];

The elements in the array allocated by new will automatically be initialized to zero. Once we have allocated an array, we can access a specific element in the array by specifying its index within the square brackets. All the indexes start at zero.

Example:

```
class Array {
    public static void main(String args[]) {
        int month_days[];
        month_days = new int[12];
        month_days[0] = 31;
        month_days[1] = 28;
        month_days[2] = 31;
        month_days[3] = 30;
        month_days[4] = 31;
        month_days[5] = 30;
        month_days[6] = 31;
        month_days[7] = 31;
        month_days[8] = 30;
        month_days[9] = 31;
        month_days[10] = 30;
        month_days[11] = 31;
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

It is possible to combine the declaration of the array variable with the allocation of the array itself, as shown here:

int month_days[] = new int[12];